Recherche dans l'espace d'états : des bases à A*

Résumé du cours AI01 — State space search (S. Toliautas), en vue de l'assignment 1.3

1. Le cadre

On cherche dans un environnement **totalement observable, discret, déterministe et statique**. La solution est donc une suite fixe d'actions, qu'on peut calculer entièrement avant d'agir.

Un problème de recherche est décrit par :

- un **état initial** s_0 ;
- une **fonction de successeurs** successors(s) → {(a, s′)} : les actions possibles et les états où elles mènent ;
- un **test de but** goal(s) → vrai / faux ;
- un **chemin** p = [s_0, a_0, s_1, a_1, …, s_g], qui alterne états et actions ;
- en option, un **coût** cost(p) et une **heuristique** h(s).

On compare les stratégies selon quatre critères : **complétude** (trouve-t-on toujours une solution si elle existe ?), **optimalité** (trouve-t-on la meilleure ?), **temps** et **mémoire**. Ils s'expriment avec b = facteur de branchement (nombre de voisins), d = profondeur de la solution et m = profondeur maximale de l'arbre (éventuellement infinie).

2. L'algorithme de base : un seul squelette pour toutes les stratégies

Toutes les recherches du cours suivent le même schéma. On garde une **frontière**, c'est-à-dire la liste des chemins dont le dernier état n'a pas encore été développé. À chaque tour :

1. on **choisit** un chemin dans la frontière ;
2. on **développe** son dernier état : on génère ses successeurs ;
3. on **ajoute** les nouveaux chemins à la frontière.

On s'arrête quand on atteint un état but, ou quand la frontière est vide (pas de solution).

> **Seule la règle de choix du chemin change d'une stratégie à l'autre.** Prendre le plus ancien donne un BFS, le plus récent un DFS, le moins cher un MCS, et le plus petit coût + h donne A*.

Deux améliorations évitent l'explosion de l'arbre de recherche :

- **Élimination des boucles** : on refuse un état déjà présent *dans le même chemin*. Un même état peut encore apparaître dans plusieurs chemins différents de la frontière.
- **Recherche en graphe** (cache d'états) : on mémorise dans un ensemble `explored` tous les états déjà rencontrés, et chaque état n'est développé qu'une fois.

La version « à coût » : Minimal-Cost Search (MCS, aussi appelée UCS)

BFS et DFS mesurent le coût d'un chemin par son nombre de pas. Pour gérer des coûts variables, le cours introduit le MCS, qui suit quatre règles :

- cost(p) doit être **strictement croissant** le long d'un chemin (coûts d'actions positifs) ;
- la frontière est **triée par coût**, et on prend toujours le chemin le moins cher ;
- pour chaque état non encore développé, on ne garde **que le meilleur chemin** qui y mène ;
- le **test de but** se fait quand on **retire** un chemin de la frontière, pas quand on le génère. On marque l'état comme exploré à ce moment-là aussi.

Voici l'algorithme du cours (slide 19), commenté. **C'est aussi l'algorithme de A*** :

```
def costsearch(model, depth):
    (start, goal, successors, cost) = model
    if goal(start): return [start]
    frontier = [ [start] ]                  # triée par cost(chemin)
    explored = set([start])
    while frontier:
        path  = frontier.pop(0)             # chemin le MOINS CHER
        state = path[-1]
        if goal(state): return path         # test au retrait => optimal
        explored.add(state)
        if len(path)//2 >= depth: continue  # limite de profondeur
        for (action, new_state) in successors(state):
            if new_state not in explored:
                new_path = path + [action, new_state]
                curr = existing_path_to(new_state)    # déjà dans la frontière ?
                if curr:
                    if cost(new_path) < cost(curr):  # on garde le meilleur
                        frontier.remove(curr)
                    else: continue
                insert_sorted(frontier, new_path, cost)
    return []
```

MCS est **complet et optimal** si les coûts des actions sont positifs. Sa complexité est $O(b^{1+C/c})$, où C est le coût de la solution et c le coût typique d'un pas.

3. A* : le même algorithme, avec une meilleure fonction de coût

L'heuristique

Une **heuristique** h(s) **estime le coût restant** de s jusqu'au but. Pour que A* fonctionne, elle doit être **optimiste** :

$$h(s) \leq \text{coût réel restant} \quad \text{(on dit que h est } \textbf{admissible}\text{)}$$

La règle de A*

A* choisit le chemin p (qui se termine en s) qui minimise :

$$\mathbf{f(p) = c(p) + h(s)} \quad \text{(coût déjà payé + estimation du reste)}$$

Le cours le dit explicitement : A* est *« identique à MCS dans son implémentation »*. On reprend l'algorithme de la section 2 et on lui donne la fonction de coût `cost(p) = c(p) + h(dernier état)`.

A* contient les autres stratégies comme cas particuliers :

- avec **h(s) = 0** (l'heuristique la plus simple, toujours admissible), on obtient f = c(p) : **A* est exactement MCS** ;
- si en plus chaque action coûte 1, c(p) est le nombre de pas : A* développe les chemins **niveau par niveau, comme BFS** ;
- si au contraire on ignore c(p) et qu'on trie par h(s) seul : c'est la recherche **gloutonne (greedy / best-first)**, qui n'est plus optimale.

Propriétés. Avec une heuristique admissible, A* est **complet et optimal**. Au pire, sa complexité reste $O(b^d)$. En pratique, une bonne heuristique réduit le facteur de branchement à un *facteur effectif* b* bien plus petit que b. Plus h est proche du vrai coût, moins A* développe d'états. Le cours illustre cette différence par les « contours » : MCS s'étend en cercles autour du départ, alors que A* s'étire vers le but.

Pourquoi l'heuristique doit être optimiste. Si h *surestime* le coût d'un bon chemin, f le fait paraître trop cher. A* peut alors retirer de la frontière un chemin vers le but plus coûteux avant d'avoir regardé le bon, et renvoyer une solution non optimale. Avec h optimiste, un chemin prometteur n'est jamais mis de côté à tort.

4. Exemple : A* sur le taquin avec h_1 (distances de Manhattan)

Construire l'heuristique en relâchant le problème

Règle du taquin : une tuile A peut glisser vers B **si A et B sont voisines** et **si B est vide**. On obtient une heuristique en **supprimant une contrainte** :

- on supprime « B est vide » : chaque tuile se déplace librement vers sa case. Le nombre exact de coups de ce problème relâché est la **somme des distances de Manhattan h_1** ;
- on supprime les deux contraintes : chaque tuile se téléporte en un coup. Le nombre de coups est alors le **nombre de tuiles mal placées h_2**.

La solution exacte d'un problème relâché ne peut pas dépasser celle du vrai problème, puisque le vrai problème a plus de contraintes. Ces heuristiques sont donc **admissibles par construction**.

Calcul de h_1

Pour chaque tuile (**sauf la case vide**), on additionne |Δligne| + |Δcolonne| entre sa position actuelle et sa position dans le but.

1	2	3	7
8	4	5	6
9		10	11
12	13	14	15

départ (`problems[1]`)

	1	2	3
4	5	6	7
8	9	10	11
12	13	14	15

but

Les tuiles 1, 2, 3, 4, 5, 6, 7, 8 et 9 sont chacune à distance 1 de leur place, et les autres sont déjà bien placées. On a donc **$h_1 = 9$**. La solution optimale stockée fait justement 9 coups : ici h_1 est exacte. Sur des puzzles plus durs, elle sous-estime mais reste un bon guide :

	h_1(départ)	longueur réelle	temps A* mesuré
`problems[1]`	9	9	instantané
`problems[5]`	26	30	0,04 s
`problems[6]`	21	35	1,0 s

Premier pas de A*

Au départ, f = 0 + 9 = 9. On développe les 4 mouvements possibles de la case vide :

mouvement	c (nb de coups)	h_1	$f = c + h_1$
L	1	8	**9**
R	1	10	11
U	1	10	11
D	1	10	11

A* prend L, le chemin dont f est le plus petit, et recommence. Un BFS (ou MCS, ce qui revient au même avec h = 0) traiterait les 4 chemins à égalité et continuerait ainsi niveau après niveau. C'est cette direction donnée par h qui fait gagner un temps exponentiel.

Implémentation dans `slide_domain_thin.py`

```
# dans __init__ : position de chaque tuile dans le but (calculée une fois)
self.goal_pos = {tile: i for i, tile in enumerate(self.goal)}

def heuristic(self, state):
    """h1 : somme des distances de Manhattan."""
    N, h = self.size, 0
    for i, tile in enumerate(state):
        if tile == SPACE: continue        # la case vide ne compte pas
        j = self.goal_pos[tile]
        h += abs(i // N - j // N) + abs(i % N - j % N)
    return h

def astar_cost(self, path):
    """f = c(p) + h(dernier état)"""
    return len(path) // 2 + self.heuristic(path[-1])
```

`lcgq` (le MCS de `state_search_thin.py`) trie sa frontière avec cette fonction. On obtient donc A* sans modifier l'algorithme de recherche.

Choisir entre plusieurs heuristiques

Pour toute position, $h_1 \geq h_2$: une tuile mal placée est au moins à distance 1. On dit que h_1 **domine** h_2 : elle est plus proche de la vérité, donc meilleure. Le cours rappelle aussi qu'il n'est pas nécessaire de choisir : $h_{opt} = \max(h_1, h_2)$ reste admissible et est au moins aussi bonne que chacune des deux.

5. Les autres stratégies du cours (sans code)

Parcours en largeur (BFS). On prend toujours le chemin *le plus ancien* de la frontière (une file FIFO). L'arbre est donc exploré niveau par niveau : tous les chemins de longueur 1, puis ceux de longueur 2, etc. La première solution trouvée est la plus courte. Le défaut est la mémoire : il faut garder tout un niveau de l'arbre, soit $O(b^d)$ chemins.

Parcours en profondeur (DFS). On prend toujours le chemin *le plus récent* (une pile LIFO). On descend le plus loin possible dans une branche avant de revenir en arrière. La mémoire est faible, O(bm), car on ne garde que la branche courante et ses voisins. En revanche, DFS peut s'enfoncer indéfiniment dans un espace infini, et la première solution trouvée n'est pas forcément la meilleure.

Approfondissement itératif (IDS). On lance un DFS limité à la profondeur 1, puis 2, puis 3, et ainsi de suite jusqu'à trouver une solution. On combine ainsi la faible mémoire du DFS avec la garantie du BFS de trouver la solution la plus courte. Refaire les premiers niveaux coûte peu, car l'essentiel du travail se trouve au dernier niveau.

Recherche bidirectionnelle. On lance deux recherches (BFS, IDS ou UCS) en même temps, une depuis le départ et une depuis le but. On s'arrête quand les deux frontières se rencontrent. Comme chaque recherche ne va qu'à mi-profondeur, la complexité devient $O(b^{d/2})$ au lieu de $O(b^d)$. Il faut en revanche pouvoir inverser les actions pour partir du but.

Recherche gloutonne (greedy / best-first). On choisit le chemin dont *l'état final semble le plus proche du but*, selon h seul, sans tenir compte du chemin déjà parcouru. Elle est souvent rapide, mais elle se laisse attirer par la direction indiquée par h comme un DFS, même si cette direction est une impasse ou un détour. Elle n'est ni complète ni optimale.

A* à mémoire bornée (MA*, SMA*). Comme A*, mais la frontière a une taille maximale. Quand elle est pleine, on supprime les chemins les plus chers (et les plus anciens) en mémorisant leur coût, puis on les

régénère plus tard si la frontière se vide. L'algorithme reste complet et optimal tant que la solution tient en mémoire.

IDA* et RBFS. Ce sont des variantes de A* qui utilisent encore moins de mémoire. IDA* reprend l'idée de l'IDS, mais la limite porte sur f = c + h au lieu de la profondeur. Comme toujours, économiser la mémoire coûte du temps de calcul.

Récapitulatif

Stratégie	Chemin choisi	Complet	Optimal	Temps / mémoire
BFS	le plus ancien	oui	oui [1]	$O(b^d)$ / $O(b^d)$
DFS	le plus récent	non	non	$O(b^m)$ / $O(bm)$
IDS	DFS à profondeur croissante	oui	oui [1]	$O(b^d)$ / $O(bd)$
Bidirectionnelle	deux recherches qui se rejoignent	oui	oui	$O(b^{d/2})$
MCS / UCS	plus petit c(p)	oui	oui	$O(b^{1+C/c})$
Greedy	plus petit h(s)	non	non	$O(b^m)$
A*	**plus petit c(p) + h(s)**	**oui**	**oui [2]**	$O(b^d)$, en pratique bien moins

[1] si toutes les actions coûtent pareil [2] si h est admissible